

Créer un Joueur d'Échecs avec l'Apprentissage par Renforcement

L'objectif est de développer A à Z un agent capable d'apprendre à jouer aux échecs en partant de zéro, uniquement en jouant contre lui-même. Ce domaine, popularisé par des projets comme AlphaZero de DeepMind, est à la pointe de l'intelligence artificielle.

Ce guide est structuré en **Work Packages (WP)**, chacun représentant une étape clé du projet. L'objectif est de leur fournir une feuille de route claire, de la théorie à la mise en œuvre pratique.

WP 1 : Mise en Place de l'Environnement et des Interactions

Objectif : Créer un environnement de jeu fonctionnel où un agent peut effectuer des actions (coups) et recevoir des retours (état du jeu).

Tâches Clés :

1. **Choisir une bibliothèque d'échecs :** Intégrez une bibliothèque Python robuste pour gérer la logique du jeu. **python-chess** est un excellent choix. Elle permet de :
 - Générer les coups légaux depuis une position donnée.
 - Modifier l'état du plateau.
 - Détecter les fins de partie (échec et mat, pat, nulle).
2. **Représenter l'état du jeu :** L'état du jeu (la position sur l'échiquier) doit être transformé en un format numérique que le réseau de neurones peut comprendre. Une approche courante est d'utiliser une série de "plans binaires" (bitboards). Par exemple, un tenseur de dimension (8, 8, N) où N est le nombre de caractéristiques.
 - *Exemples de plans :* un plan pour les pions blancs, un pour les fous noirs, un plan indiquant si le roque est possible, etc.
3. **Implémenter la boucle de jeu :** Créez une fonction ou une classe qui gère une partie complète. L'agent choisit un coup, l'environnement se met à jour, et on recommence jusqu'à la fin de la partie.

Concepts Théoriques à Maîtriser :

- **Processus de Décision Markovien (MDP) :** Formaliser le jeu d'échecs comme un MDP (États, Actions, Récompenses).
- **Représentation d'état (State Representation).**

Livrable : Un script Python capable de simuler une partie d'échecs entre deux joueurs choisissant des coups aléatoires parmi les coups légaux.

WP 2 : Le Réseau de Neurones

Objectif : Concevoir et implémenter l'architecture du réseau de neurones qui servira de "cerveau" à notre agent.

Tâches Clés :

1. **Architecture du réseau** : Le réseau prendra en entrée la représentation de l'état du jeu (du WP 1) et produira deux sorties :
 - **Une tête "Politique" (Policy Head)** : Un vecteur de probabilités sur tous les coups possibles. Il nous dit *quel coup jouer*.
 - **Une tête "Valeur" (Value Head)** : Un scalaire unique entre -1 et 1, estimant la probabilité de victoire depuis la position actuelle. Il nous dit si la *position est bonne ou mauvaise*.
2. **Inspiration d'AlphaZero** : L'architecture classique utilise un "corps" de couches convolutives (pour analyser la structure spatiale de l'échiquier) suivi de deux têtes de sortie distinctes (couches entièrement connectées).
3. **Implémentation** : Codez cette architecture avec PyTorch. Assurez-vous que les dimensions d'entrée et de sortie correspondent bien à votre représentation d'état et à l'espace d'actions.

Livrable : Une classe `NeuralNetwork` qui peut effectuer une passe avant (forward pass) : prendre un état du jeu et retourner une politique et une valeur.

WP 3 : L'Algorithme de Recherche - Monte Carlo Tree Search (MCTS)

Objectif : Implémenter l'algorithme MCTS, qui permet à l'agent de "réfléchir" et de choisir le meilleur coup en explorant les futurs possibles. Le MCTS est guidé par le réseau de neurones.

Tâches Clés :

1. **Comprendre le MCTS** : L'algorithme se déroule en quatre étapes répétées en boucle pour chaque coup à jouer :
 - **Sélection** : Partir de la racine (position actuelle) et descendre dans l'arbre de jeu en choisissant les nœuds les plus prometteurs (guidé par une formule comme UCB1).
 - **Expansion** : Lorsqu'on atteint un nœud "feuille", l'étendre en ajoutant ses enfants (les positions suivantes) à l'arbre.
 - **Simulation (Rollout)** : Évaluer le nœud feuille avec la tête "Valeur" du réseau de neurones. C'est ici que le réseau guide la recherche.
 - **Rétropropagation (Backpropagation)** : Mettre à jour les statistiques (nombre de visites, valeur accumulée) des nœuds sur le chemin parcouru.
2. **Implémenter la structure de l'arbre** : Créez une classe `Node` pour représenter chaque position dans l'arbre de recherche. Chaque nœud doit stocker des informations essentielles : sa position, le coup qui y a mené, le nombre de visites (N), et la valeur totale (W).
3. **Connecter MCTS et le réseau** : Le réseau de neurones est utilisé à deux moments cruciaux :
 - La tête **Politique** guide la phase de **Sélection** en fournissant des probabilités a priori sur les meilleurs coups.
 - La tête **Valeur** fournit l'évaluation lors de la phase de **Simulation**.

Livrable : Une fonction MCTS(position, neural_network) qui, après un certain nombre de simulations, retourne la distribution de probabilités des coups à jouer depuis la position. Le coup joué est ensuite sélectionné à partir de cette distribution.

WP 4 : La Boucle d'Auto-Apprentissage (Self-Play) et l'Entraînement

Objectif : Mettre en place la boucle qui permet à l'agent de s'améliorer continuellement : il génère des données en jouant contre lui-même, puis s'entraîne sur ces données.

Tâches Clés :

1. Génération de données (Self-Play) :

- Simulez des milliers de parties de l'agent contre lui-même.
- Pour chaque coup, exécutez l'algorithme MCTS (WP 3) pour décider du coup à jouer.
- Stockez l'historique de chaque partie : (état, distribution de probabilités MCTS, résultat final de la partie). Le résultat est +1 pour une victoire, -1 pour une défaite, 0 pour une nulle.

2. Préparation des données pour l'entraînement : Les données brutes stockées doivent être formatées pour l'entraînement du réseau. Chaque exemple d'entraînement sera une paire :

- **Entrée :** Un état du jeu.
- **Cible :** La distribution de probabilités issue du MCTS (pour la tête Politique) et le résultat final de la partie (pour la tête Valeur).

3. Phase d'entraînement :

- Utilisez les données générées pour entraîner le réseau de neurones (WP 2).
- La **fonction de perte** sera une combinaison de deux termes :
 - Perte pour la tête Politique : Divergence de Kullback-Leibler ou entropie croisée entre la sortie du réseau et la distribution MCTS.
 - Perte pour la tête Valeur : Erreur quadratique moyenne entre la valeur prédite et le résultat réel de la partie.
- Entraînez le modèle pendant plusieurs époques.

Livrable : Un script principal qui orchestre la boucle : génération de données -> entraînement -> mise à jour du réseau -> nouvelle génération de données.

WP 5 : Évaluation et Itération

Objectif : Évaluer objectivement la force de l'agent et s'assurer qu'il progresse.

Tâches Clés :

1. Mise en place d'un tournoi d'évaluation :

- Après chaque cycle d'entraînement, opposez le **nouveau modèle** à l'**ancien modèle**.

- Faites-les jouer un certain nombre de parties (ex: 100).
- Si le nouveau modèle gagne avec un pourcentage significatif (ex: > 55%), il devient le nouveau "meilleur" modèle pour la prochaine phase de génération de données. Sinon, on garde l'ancien.

2. Mesurer la force de jeu :

- Faites jouer votre agent contre des moteurs d'échecs existants à différents niveaux (comme Stockfish avec une profondeur de recherche limitée).
- Utilisez des protocoles standards comme le **Universal Chess Interface (UCI)** pour faciliter ces confrontations.

3. Analyse et débogage :

- Visualisez les parties jouées. L'agent fait-il des erreurs évidentes ? Comprend-il les concepts de base (valeur des pièces, contrôle du centre) ?
- Tracez l'évolution de la fonction de perte au fil du temps.

Livrable : Un framework d'évaluation robuste et des graphiques montrant la progression de la force de l'agent au fil des cycles d'entraînement.

Chronologie et Conseils

- **Gestion du temps :** Ce projet est ambitieux. Le WP 1 et 2 sont les fondations. Le WP 3 (MCTS) est souvent le plus complexe à implémenter correctement. Le WP 4 demande beaucoup de temps de calcul.
- **Commencer simple :** Avant de vous lancer dans les échecs, vous pouvez tester vos algorithmes (MCTS, réseau) sur des jeux plus simples comme le **Morpion** ou **Puissance 4**. Cela permet de déboguer beaucoup plus rapidement.

Evaluation du projet :

- Soutenance mi-parcours 12/01/26 (30%)
- Soutenance et compétition le 27/02/26 (40%)
- Rapport à rendre le 27/02/26 (30%)